DATA.TXT FILE

In our project, players control a new intern competing for a return offer by gaining favor with the boss and undermining other interns. The game's core mechanics involve collecting items throughout the office that can be used strategically to either impress the boss or sabotage competitors. Interactions with NPCs, including the boss and the secretary, allow players to gather clues and events that can improve their standing or create opportunities for mischief. Actions like throwing coffee, hacking computers, or sneaking around the office to gather secrets are all part of the strategy.

The game world consists of rooms, items, and characters, with certain mechanics allowing for object interactions, status effects, and event triggers; all read and stored in the data.txt file. Commands to the player are sent as keyboard input which is parsed and mapped with our data file to figure out the right verb, object (and even a preposition which relates another object) to the right subroutine or action call. The behaviours, actions, including object attributes modification is done in code. Thus the interface will be text-based similar to handy unix tools that are run in the terminal. The Data file consists of 4 main sections: Objects, Commands, Variables, and Event dialogues. The table 1 below gives a brief overview.

Data section	**Use case**	**Examples**
Objects	Objects (areas/rooms, items, and characters) are stored in this data section as a dictionary with unique names for each object together with the attributes it has. The common attributes include the object's holder, objects it is holding, its description, and the object type. For areas/rooms we have the destination to its North, South, East, West, and this is done for navigation in the game. For items we have attributes such as "energize" "poison" "emits_light". For characters we have the status effects and the character name.	Objects["cafeteria"]["description"] to access the cafeteria room and its room description: "A small comfortable environment to enjoy meals or snacks during break. Variety of food options can be bought here." "Secretary_office": { … "NSEW": ["boss_office", "main_lobby",null,null]} `> go south` go to current room's destination to the south, to main_lobby `> drink coffee` will make game speed faster for the consumer What is your name player? `> gord` puts as the player name

Commands	Groups verbs into synonyms which maps to the same command i.e. id 4 that maps to "grab", "get", "take", "pick up" which all correspond to the player/NPC putting the item on his inventory.
	
	Including the Articles ["of", "a", "an", "the"] so that will be parsed out of the player/NPC input.
	
	Prepositions ["at","in","above","under"] are for tokenizing user/npc input for distinct subroutine calls. Making actions in relation to another object.
	
	Lookup table for mapping the “verb” “object” “preposition” “object” to a subroutine call.
		"4": ["grab","get","take","pick up"],
		> grab coffee is the same as
		> take coffee
		
		> grab the coffee becomes just
		> grab coffee
		
		`> throw coffee at Philip`
		distinguishes where to throw at
		
		[14,null,null,null,"show_room"],
		[6,"room",null,null,"show_room"],
		`> look around` does the same call
		`> examine room`

Events	The data here will contain the dialogues for each of the game events that will be encountered.	e.g. at the beginning of the game the “greet_at_game_start” dialogue will be read and have the game introduced with the setting of the building, characters in the game, and the duty needed to succeed.
Variables	The game constants and relevant variables to keep track of for the game. This includes the speed that the player or NPC will be going at a pace. Number of turns to skip when a forced event happened.	`world_speed = 100` the default, 120 is energized, 70 is poisoned

Table 1.0

More about objects

An object in our game context is either a room, an item, or a character. Each object is a “container” meaning a room can hold a character or item, a character can hold an item, but even a “bottle” item can hold items. There is no real difference between a room and an item as far as the game is concerned. They can both hold things, heck even a player can be trapped inside an item like a chest or a player can carry another character. The “holder” and “holding” then are applicable in all these objects. Though if any checks are necessary, we have the “type” attribute to distinguish an object being an area, a character, or an item.

Only the “holder” attribute is needed to be written in the file as the “holding” objects are automatically generated based on the holder. We can simulate object duplicity by defining the “holding” attribute on our own in the data file to include the items since this can be a necessity at times; for example tables, cabinets, doors, and shelves can be in multiple rooms and the auto-generation assumes only one ownership of the object.

Each character will contain a “holder” which the subroutine will use to determine the location of the main playable characters and other NPC’s. For example, the player will spawn in the main lobby and therefore the holder of the player will be main_lobby. When the player moves to a different room the holder will change and update to the current room.

Each player will hold a “likeability” attribute as an integer number. Each time the boss positively interacts with a character the value will go up and when the boss negatively interacts with the character the value will go down. At the end of the game, we will use a function to compare each of the characters (player/NPC 1/NPC 2) Likeability score to determine which of the possible endings will take place.

Another attribute to a character is “friendliness” which will measure the friendliness score the player will get with his fellow interns, in other words, how the NPC’s will behave towards the player. Yes, the NPC’s do not act differently with other NPC’s, just towards the player. Aside from behaviour, this attribute also affects what ending will be achieved e.g. the ending where we all (player and NPC’s) get a return offer in the end.

Object change in some cases will be done in a pop-in-pop-out fashion. For example coffee and laxative to poisoned_coffee. A function will wait for the predetermined command “combine” from a user input. This command will run in the subroutine to check if the player/NPC has the acceptable items to enact a combination. After the function has determined that they have the prerequisite items the function will change the value of the the two prerequisite items to null in the players/NPC inventory and then add the new combination item into the inventory in its stead. For example, the laxative coffee need coffee and laxatives and will destroy the previous item and deposit the item laxative_coffee into the inventory

Game Files

The project consists of 3 files as of Feb 16th 2025. Where the game will be worked on, on separate parts for each group member. Jan implemented a demo of the game, currently being able to demonstrate the “hold” and “holder” attribute to navigate the game world and interact with objects by pickup and drop (including getting NPC into inventory). Jan is to work on the NPC AI implementation. Jonathan is to work on the parser, where user/npc input will be mapped to a certain subroutine and of course processing that input. Jake and Jaeden are to work on the subroutines where the character behaviour, world interaction, and events.

data.txt - Defines the game world, including rooms, items, characters, and verbs for interaction.
jan_subroutines.py - Contains Python functions to process game logic, player actions, and navigation.
Project_demo.py - collectively unites the code to call on different files to run the game

File Overview of data.txt

a) Areas or Rooms

Screen is a special object in the game to tell where the current things can be seen or viewed. In other words, when switching view to control the NPC's, the big events are shown with respect to the screen and the other things happening in the background e.g. big character interaction that changes the story is now shown with the NPC's perspective.

Rooms are defined with:
Type "room"
Room description
Holding or the items being held, this is optional
Holder if screen "holds" it
NSEW Navigation defines movement directions e.g. "NSEW": [null, null, "cafeteria", null] means the cafeteria is to the east

Example:

```
"main_lobby": {
    "type": "room",
    "description": "The central entrance area, designed to welcome visitors and employees.",
    "NSEW": ["secretary_office", null, null, "cafeteria"]
}
```
To the north is the secretary_office
To the west is the cafeteria
No paths east or south

b) Items

Each has:
Type "item"
Item description
Holding if contains items e.g. envelop holding two papers, this is optional
Holder which shows the character or room that has it
Attributes that affects world interaction like "energize", "poison", "emits_light"

Example:

```
"coffee": {
    "holder": null,
    "description": "A cup of coffee that increases energy and focus.",
```

"attributes": ["energize", "friendliness","consumable”]
}
Can be picked up and consumed.
Can be modified (e.g., adding a laxative)

c) Characters

Players and NPCs have:

Likability affecting
Friendliness for NPC to behave differently and affects world ending
Holding is what objects is being held, this is optional
Holder which is an area in the game world
Status effects such as "healthy", "poisoned", "hacked", etc.

Example:

"NPC_1": {
"likability": 0,
"friendliness": 2,
"holder": "washroom",
"status": ["poisoned", "hacked"]
}
Neutral NPC (friendliness: 2).
Currently poisoned ("status": ["poisoned"]).

d) Verbs

Defines commands the player can use and input. Each verb has multiple synonyms for ease of use, which are mapped to numbers for execution (eg. {0: ["N", "north", "go north"]}.

Examples:

Verb	Function
"N", "north", "go north"	Move north
"grab", "get", "take"	Pick up an item
"drop", "put down"	Drop an item
"consume", "drink", "use"	Use an item

"sabotage"	Harm another character
"hug", "give"	Positive interactions

Table 2.0

e) Events

Things which can take place throughout the game, for example, a greeting at the beginning of the game, or a specific ending which is reached after meeting certain conditions.

"Events": {
"greet_at_game_start": "Hey there, welcome to the team! We're excited to have you on board for your internship. It's a great opportunity to learn and grow, and we're looking forward to seeing the amazing contributions you'll make. ",
"throw_coffee_at_intern": "WTF man!!! what's wrong with you, now I have to go to the washroom...",
"ending_1": "You married the boss's daughter, now you get to inherit the company."
}

f) Lookup Table

Maps the user input to subroutine.

This is small part of the lookup table:

```
"lookup_table": [
    //[verb, direct_obj, preposition, indirect_obj, subroutine_key]
    [0,null,null,null,"go_direction"],
    [8, "coffee",null,null,"energize_character"],
    [8,"*",null,null,"eat_inedible"],
    [9,"coffee","at","*","thrown_coffee_at_x"],  //throw
  ]
```